
Mind Your Own Target: Revisiting Uncertainty Quantification in Offline Reinforcement Learning

Vít Unčovský Václav Nevyhoštěný Petr Novotný
Masaryk University, Faculty of Informatics petr.novotny@fi.muni.cz

Abstract

Model-free offline RL methods typically utilize multiple critics to mitigate overestimation bias for out-of-distribution (OOD) actions: this includes both the standard twin-critic setup and more extensive critic ensembles capturing the epistemic uncertainty about the value estimates. In the latter group, the foundational methods (such as SAC-N and EDAC) only handle epistemic uncertainty implicitly, taking the worst-case aggregation (WA), i.e., a minimum, of critic predictions. More recent uncertainty-quantifying (UQ) methods, such as PBRL and MSG, explicitly compute some measure of intra-ensemble disagreement (e.g., the variance of predicted values), which is used to modify the actor or critic losses. As these newer methods have not yet been included in principled open-source evaluations of offline RL algorithms, the effects of explicit uncertainty quantification in offline RL remain poorly understood. In this paper, we present a thorough and methodical evaluation of two key UQ methods, MSG and PBRL, against a range of baselines across a diverse set of D4RL tasks. The evaluation shows that the UQ methods fail to consistently outperform the WA methods, but also that all of the considered ensemble methods are outperformed by double-critic policy-constrained methods on most tasks, thus extending a previous observation about WA methods to the more recent UQ approaches. Interestingly, the evaluated UQ methods outperform WA ones on several important tasks, and our ablations suggest that this is due to the former using independent bootstrap targets for each critic (thus confirming previous theoretical criticism of shared targets). We demonstrate that ensembles with independent targets are a key design choice and that even policy-constrained methods can substantially benefit from their adoption.

1 Introduction

Design motifs in model-free offline RL algorithms In offline reinforcement learning [15], a policy is trained on a dataset of pre-collected trajectories without any interactions with the environment. The offline RL algorithms need to account for the shift between the state-action distribution of the dataset (induced by some unknown behavior data-collection policy) and the distribution encountered when deploying the trained policy in the environment. While a variety of approaches dealing with this distribution shift were developed, they build on (and combine) several underlying design motifs, such as *policy-constrained* training (regularizing the actor towards the behavior policy [6]) or direct penalization of out-of-distribution (OOD) action values [13]. One of the most recurrent motifs in offline RL setting is the use of multiple critics to mitigate the overestimation bias of OOD actions. Indeed, already the most prominent policy-constrained methods [19] utilize a twin-critic setup. However, there is a large class of offline RL methods which presume the usage of much larger ensembles of critics (dozens, or even hundreds) to capture the epistemic uncertainty about value estimates: we refer to these as (proper) *ensemble* methods in this paper.

Ensemble methods in offline RL We further broadly categorize the ensemble methods into two sub-classes based on how they express the pessimism about the value estimates.

- *Worst-case aggregation (WA)* methods, such as SAC-N and EDAC [1] aggregate the value estimates by taking the most pessimistic prediction (minimum) over the ensemble.
- Somewhat more complex *uncertainty-quantification (UQ)*¹ approaches, such as PBRL [3] and MSG [7], explicitly compute some measure of intra-ensemble disagreement (e.g., the variance of $Q(s, a)$ -estimates over the whole ensemble) and use this measure to modify the actor or critic losses (e.g., by applying a lower confidence bound penalization).

Evaluation of ensemble methods: key limitations The main motivation behind our work is the lack of large-scale, well-designed, open-source evaluations of ensemble methods including the UQ approaches. The experimental evaluations in primary sources suffer from several limitations:

- The experimental comparison in the papers does not evaluate the algorithms across varying online hyperparameter tuning budgets, which prior work [10, 14] showed crucial for reliably comparing the methods
- Often, the hyperparameters of the presented methods are tuned individually for each environment, which makes it difficult to establish the “out-of-the-box” performance of each algorithm. [23]
- Ablations and analyses of hyperparameter sensitivity are usually performed only on a single environment and not across different tasks. [3, 21]
- The reference implementations sometimes employ various heuristics, such as reward transformations on navigation environments in [7, 13, 23] or a behavior cloning pretraining phase in [7, 23], which are not ablated in the experiments. This makes it difficult to isolate the effect of core algorithmic advances on the overall performance.

Large-scale evaluations of selected offline RL baselines on top of CORL and Unifloral libraries [10, 20] were published, covering foundational WA methods, such as SAC-N and EDAC [1]. However, these evaluations do not cover the more recent (and complex) UQ approaches, such as PBRL [3], MSG [7], or RORL [23]. Thus, the effect of explicit uncertainty estimation on offline RL performance has not been clearly established.

Our contribution We provide a thorough empirical evaluation of offline RL methods with a particular focus on ensemble methods with explicit uncertainty estimation. We compare them against a range of baselines, including both WA ensemble methods SAC-N and EDAC [1], policy-constrained methods such as ReBRAC [19], IQL, and AWAC [16]. Concretely, we present the first comprehensive evaluation of the PBRL and MSG algorithms avoiding the pitfalls highlighted above. Although newer methods based on these two exist, they typically use the same policy evaluation losses, adding additional generative components [21] or auxiliary regularization terms [23, 25]. Our aim is to evaluate the core components of the actor and critic losses across a wide range of scenarios, and hence we focus our evaluation on the two aforementioned algorithms.

We implement MSG and PBRL on top of the Unifloral library [10], using a unified implementation structure that facilitates ablation of algorithm design choices. We then perform a series of experiments on tasks from the D4RL [5] benchmark suite (MuJoCo, AntMaze, Adroit and Franka), using an evaluation procedure of [10]. Our experimental findings are as follows:

Firstly, in the comprehensive evaluation over all the tasks (Section 4.1), the newer methods with explicit uncertainty quantification do not significantly outperform SAC-N or EDAC, suggesting that their architectural additions offer limited gains. Importantly, all the considered ensemble methods are significantly outperformed by behavior cloning-based baselines (confirming similar finding limited to WA methods in [10]), highlighting the question on the sufficiency of ensemble uncertainty as a form of regularization.

Secondly, despite the overall non-superiority of the studied UQ methods over WA ones, we find that the former do significantly outperform the latter on the Adroit and AntMaze tasks. The failure of

¹While the minimum operator in WA methods can also be seen as a proxy for an uncertainty signal, we reserve the UQ class for methods that explicitly compute some disagreement statistic.

SAC-N and EDAC to obtain any rewards on the sparse-reward **antmaze** tasks has been acknowledged in prior evaluations [20, 25], but has to our best knowledge never been thoroughly analyzed. Through a series of ablations in Section 4.2, we identify independent bootstrap targets as the key choice distinguishing MSG and PBRL from the failing baselines, providing a robust confirmation of the theoretical analysis of shared vs. independent targets from [7].

Thirdly, to further explore and validate the importance of independent targets, we propose two new algorithms combining the strongest baselines in the overall evaluation (the policy-constrained methods AWAC and ReBRAC) with larger independent-target ensembles. The new algorithms, U-AWAC and U-BRAC, are evaluated in Section 4.3. We find that independent targets yield a qualitative improvement on sparse-reward tasks, with overall performance remaining comparable to the original algorithms.

While [7] argued that sharing targets causes overestimation and ablated independence with a single regularizer, we refine the bias profile across tasks (Fig. 1b) and several regularizers in a unified implementation (Table 2), showing the benefits transfer to the best policy-constrained methods (Fig. 2 and 3).

2 Related work

Offline RL Offline RL methods address the distributional shift problem by incorporating pessimistic regularization terms. Policy-constrained methods, such as TD3+BC [6] or BRAC [22], anchor the learned actor to the behavior policy by augmenting the actor objective with a behavioral cloning (BC) regularization term. Similarly, AWAC [16] weights the BC term with exponentiated advantages, driving the agent towards the rewarding regions of the dataset. Alternatively, methods such as CQL [13] and IQL [12] directly penalize the learned value estimates, with CQL using a pessimistic Bellman operator and IQL restricting policy evaluation to in-sample actions. Finally, ensemble methods [1, 15] train an ensemble of critics [17], leveraging their predictive variance to detect and penalize unreliable actions. In [1], the authors show that increasing the critic ensemble size stabilizes Soft Actor-Critic training (SAC-N), and further propose EDAC, which adds a diversity regularizer to reduce the ensemble size required to solve certain tasks. Both methods rely only on using the pessimistic minimal ensemble prediction as a regularizer during training.

Uncertainty-quantifying methods Uncertainty-quantifying methods, such as MSG [7] and PBRL [3], introduce regularization terms that explicitly decrease the critic values. DURL [21] extends PBRL by training a trajectory density model that applies regularization only to a subset of the dataset, while RORL [23] augments the critic loss with a term that encourages smoothness of the learned estimates. UWMSG [24] extends MSG by weighing the critic loss in each state by the variance of the learned value estimates.

Evaluating Offline RL Methods Reliable evaluation of offline algorithms has received increasing attention, with works [10, 14] proposing evaluation protocols enabling robust and fair analysis. Our work directly builds on [10], extending their evaluation efforts to uncertainty-based algorithms.

3 Background

Offline RL We follow the standard formalization of offline reinforcement learning via Markov Decision Processes (MDPs) [18]. An MDP $\mathcal{M}$ is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \mu_0)$, where $\mathcal{S}$ is a state space, $\mathcal{A}$ is a set of available actions, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{D}(\mathcal{S})$ is a probabilistic transition function, $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is a reward function, and $\mu_0 \in \mathcal{D}(\mathcal{S})$ is an initial state distribution. We consider memoryless stochastic policies of the form $\pi : \mathcal{S} \rightarrow \mathcal{D}(\mathcal{A})$ that map states to distributions over actions. Each such policy π induces a distribution over trajectories $\tau = (s_1, a_1, r_1, s_2, a_2, r_2, \dots)$, where $s_1 \sim \mu_0$, $a_t \sim \pi(\cdot|s_t)$, and $s_{t+1} \sim \mathcal{P}(\cdot|s_t, a_t)$ for all t . We use $\tau \sim \pi$ to denote a trajectory τ generated by following the policy π in the MDP $\mathcal{M}$. The goal is to find a policy π that maximizes the expected return defined as $J(\pi) = \mathbb{E}_{\tau \sim \pi} [\sum_{t=1}^{\infty} \gamma^{t-1} r(s_t, a_t, s_{t+1})]$, where $\gamma \in [0, 1)$ is a discount factor.

In the case of *offline reinforcement learning*, the agent is not allowed to interact with the environment directly. Instead, it is given a fixed dataset $\mathcal{T} = \{(s_i, a_i, r_i, s_{i+1})\}_{i=1}^N$ of transitions collected by

some behavior policy π_β . The agent must learn a policy π that maximizes the expected return $J(\pi)$ using only the information contained in the dataset $\mathcal{T}$, and thus it cannot estimate the expectations precisely.

Unifying Ensemble Methods We introduce a unified framework that subsumes the ensemble methods considered in this work. We consider model-free ensemble methods that extend SAC [8], utilizing a stochastic actor π_ϕ and a critic ensemble $\mathcal{E} = \{Q_{\theta_i}\}_{i=1}^N$, each with a delayed target network θ'_i . We denote the ensemble mean and standard deviation as $\mu_{\mathcal{E}}(s, a)$ and $\sigma_{\mathcal{E}}(s, a)$, substituting $\mathcal{E}'$ for statistics computed over target networks.

The critic loss $\mathcal{L}_C$ is composed of a value loss $\mathcal{L}_V$ and a critic regularizer term $\mathcal{R}$ with strength controlled by a parameter λ_C . The actor loss $\mathcal{L}_A$ uses an aggregation operator to construct a conservative value estimate Q_{cons} , guiding the actor towards actions that maximize this estimate, augmented with an entropy bonus. Throughout, (s_t, a_t, r_t, s_{t+1}) denotes a transition from the offline dataset $\mathcal{T}$, and $\hat{a}_t$ an action sampled from the current policy $\pi_\phi(s_t)$. Overall, we write

$$\mathcal{L}_C(\theta_i) = \mathcal{L}_V(\theta_i) + \mathcal{R}(\theta_i; \lambda_C), \quad \mathcal{L}_A(\phi) = -Q_{\text{cons}}(s_t, \hat{a}_t) - \alpha \cdot \mathcal{H}(\pi_\phi(\cdot | s_t)), \quad (1)$$

where α controls the strength of entropy regularization.

The most common choice for Q_{cons} is to take the minimum Q-value across the critic ensemble, but certain UQ algorithms, including MSG, introduce an additional actor conservativeness parameter λ_A and utilize a softer LCB operator.

$$Q_{\text{cons}}(s_t, \hat{a}_t) = \begin{cases} \min_i Q_{\theta_i}(s_t, \hat{a}_t) & (\text{min}) \\ \mu_{\mathcal{E}}(s_t, \hat{a}_t) - \lambda_A \sigma_{\mathcal{E}}(s_t, \hat{a}_t) & (\text{LCB}) \end{cases} \quad (2)$$

The value loss $\mathcal{L}_v(\theta_i)$ for the i -th critic is defined as the MSE between critic predictions and a detached bootstrap target $v_{t+1}^{(i)}$. WA methods aggregate predictions across the entire ensemble to construct a *shared* bootstrap target for all critics, coupling their training. In contrast, UQ methods assign each critic Q_{θ_i} an *independent* target derived solely from its own target network, with PBRL additionally injecting pessimism into policy evaluation by penalizing this target:

$$v_{t+1}^{(i)} = \begin{cases} \min_j Q_{\theta'_j}(s_{t+1}, \hat{a}_{t+1}) & (\text{shared}) \\ Q_{\theta'_i}(s_{t+1}, \hat{a}_{t+1}) - \beta \sigma_{\mathcal{E}'}(s_{t+1}, \hat{a}_{t+1}) & (\text{indep.}) \end{cases} \quad (3)$$

The action $\hat{a}_{t+1} \sim \pi_\phi(\cdot | s_{t+1})$ is shared for all targets, and $\beta \geq 0$ is a hyperparameter modulating the value pessimism. The target is augmented with an entropy bonus $\hat{v}_{t+1}^{(i)} = v_{t+1}^{(i)} + \alpha \cdot \mathcal{H}(\pi_\phi(\cdot | s_{t+1}))$, yielding the final value loss:

$$\mathcal{L}_V(\theta_i) = \left(Q_{\theta_i}(s_t, a_t) - r_t - \gamma \hat{v}_{t+1}^{(i)} \right)^2. \quad (4)$$

Finally, the critic regularizer $\mathcal{R}(\theta_i)$ is used by EDAC to encourage intra-ensemble diversity, whereas UQ methods use explicitly pessimistic terms to mitigate the critics' extrapolation errors.

$$\mathcal{R}(\theta_i; \lambda_C) = \begin{cases} \frac{\lambda_C}{N-1} \sum_{j \neq i} \langle \nabla_a Q_{\theta_i}, \nabla_a Q_{\theta_j} \rangle & (\mathcal{R}_{\text{EDAC}}) \\ [Q_{\theta_i}(s_t, \hat{a}_t) - \text{stopgrad}(Q_{\theta_i}(s_t, \hat{a}_t) - \lambda_C \cdot \sigma_{\mathcal{E}}(s_t, \hat{a}_t))]^2 & (\mathcal{R}_{\text{PBRL}}) \\ \lambda_C (Q_{\theta_i}(s_t, \hat{a}_t) - Q_{\theta_i}(s_t, a_t)) & (\mathcal{R}_{\text{MSG}}) \end{cases} \quad (5)$$

A summary of the instantiation of all four considered ensemble methods evaluated within the unified framework, along with the hyperparameters used by each algorithm, is provided in Table 1.

Table 1: Instantiation of the four ensemble methods within the unified framework. Hyperparameters excluding ensemble size N are listed next to the method names. Regularizers are defined in Eq. 5.

Worst-case Aggregation (WA)				Uncertainty Quantification (UQ)			
method	targets	Q_{cons}	$\mathcal{R}$	method	targets	Q_{cons}	$\mathcal{R}$
SAC-N	shared	min	$\emptyset$	PBRL (β, λ_C)	indep.	min	$\mathcal{R}_{\text{PBRL}}$
EDAC (λ_C)	shared	min	$\mathcal{R}_{\text{EDAC}}$	MSG (λ_A, λ_C)	indep., $\beta=0$	LCB	$\mathcal{R}_{\text{MSG}}$

4 Experiments

Environments We consider the same nine environments as Jackson et al. [10], selecting **halfcheetah-medium-expert**, **hopper-medium**, and **walker2d-medium-replay** from the MuJoCo locomotion tasks, covering a range of suboptimal to near-expert data quality. From the Adroit suite, we select **pen-human**, **pen-cloned**, and **pen-expert** – high-dimensional manipulation tasks featuring narrow expert behavioral distributions. Finally, we include **antmaze-large-diverse**, **kitchen-mixed**, and **maze2d-large**, which necessitate composing (stitching) demonstrations; the first two further require long-horizon planning with sparse reward feedback. The observation and action space dimensions of each task, along with the decision horizon, are summarized in Table 3 in the appendix.

Evaluation Protocol To facilitate a fair comparison of the methods across a number of varied online evaluation budgets, we use the evaluation protocol introduced by [10]. We associate each method with a fixed hyperparameter space $\mathcal{H}$ shared across all tasks. Evaluation proceeds in two phases. First, we train a population of $P = 20$ policies (each trained for 1 million learning steps) with hyperparameters uniformly sampled from $\mathcal{H}$. Second, we simulate hyperparameter tuning by running a UCB bandit algorithm [2]. Concretely, we use the UCB algorithm to select the best-performing trained policy based on a fixed budget of online policy rollouts. Finally, repeating this tuning procedure on multiple random subsamples of $K \leq P$ policies from the trained population yields a bootstrap confidence interval [4] for the mean performance of a tuned algorithm for the given budget.

Hyperparameter Spaces For SAC-N, EDAC, and policy-constrained baselines excluding AWAC, we use the spaces used by the authors of Unifloral [10]. For AWAC, we adopt the hyperparameters used in the CORL library [20]. For MSG and PBRL, we consider the spaces obtained by taking the union of hyperparameter settings for which the authors report results, with $N \in \{10, 30\}$.

Research Questions We structure our evaluation around the following three research questions:

Q1: How do MSG and PBRL compare to prior baselines across tasks with different characteristics? We compare MSG and PBRL to prior baselines and find that their improvements over earlier ensemble methods are largely confined to sparse-reward settings.

Q2: Which design choices are key for solving sparse-reward tasks with ensemble algorithms? Through controlled ablations, we identify training the critic ensemble with independent bootstrap targets as the crucial design decision, confirming the theoretical insights of [7].

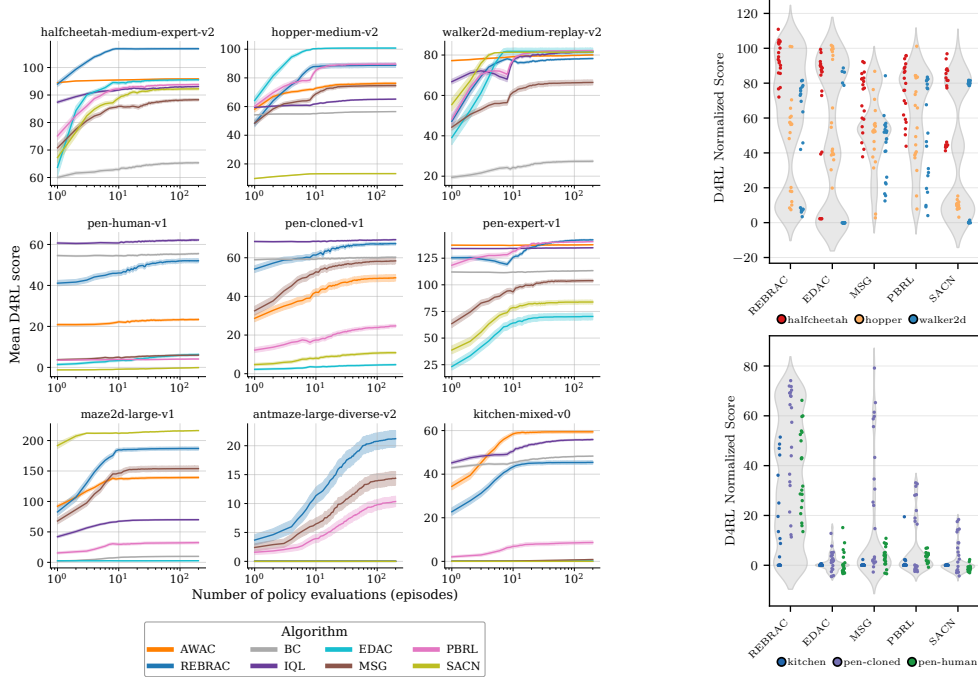
Q3: Can independent targets improve state-of-the-art policy-constrained methods? Based on our analysis, we propose two novel methods, U-AWAC and U-BRAC, utilizing the independent targets in the strongest baselines of our evaluation.

4.1 How do MSG and PBRL compare to prior baselines?

To answer the first question, we evaluate the performance of the uncertainty-quantifying (UQ) ensemble methods PBRL and MSG across the diverse set of environments introduced in the previous section. We compare their performance against foundational ensemble methods SAC-N and EDAC, which rely solely on worst-case aggregation (WA) of value estimates. For non-ensemble baselines, we also include a behavior cloning agent and the three strongest methods identified in prior evaluations [20]: the policy-constrained algorithms AWAC, ReBRAC, and IQL.

Results in Figure 1a show that neither MSG nor PBRL is competitive with the best policy-constrained methods, with ReBRAC dominating both methods across all tuning budgets on 6 of the 9 considered tasks. On the three remaining tasks **hopper**, **walker2d**, and **maze2d**, which feature dense rewards and highly suboptimal datasets, ensemble methods yield the strongest policies. However, even on these tasks, the prior WA methods of EDAC and SAC-N outperform both MSG and PBRL across most tuning thresholds, with ReBRAC trailing the best ensemble method by at most 17% in normalized score points. In contrast, on the high-dimensional expert tasks of **pen-human**, **pen-cloned**, and **kitchen**, all four ensemble methods perform notably poorly, failing to reach the performance of policy-constrained baselines and BC.

We further analyzed the behavior of ensemble methods on the challenging expert tasks, finding that all four exhibit severe hyperparameter instability: the majority of runs fail to learn meaningful



(a) The x-axis shows the hyperparameter tuning budget; the y-axis shows the mean value of a tuned policy and its bootstrap 95% confidence interval.

(b) Score distributions across all 20 runs on selected benchmark sets. Ensemble methods suffer from severe instability on expert tasks, but not on easier MuJoCo locomotion tasks.

Figure 1: Full evaluation results. Subfigure (a) shows UCB tuning performance as a function of online budget, while (b) shows distributions of the scores across runs, revealing instability of ensemble methods on expert tasks.

policies, while only a small fraction succeed. We visualize the distributions of trained policy scores across different benchmarks in Figure 1b. Notably, prior ablation analyses, including those of PBRL, RORL, and DURL [3, 21, 23], are conducted exclusively on the MuJoCo locomotion environments, where training of these methods is far more stable. Our findings further highlight the need to perform hyperparameter ablations of offline algorithms across a representative range of tasks with diverse properties.

While the overall evaluation is dominated by policy-constrained baselines, the head-to-head comparison between WA and UQ methods is more subtle. As mentioned above, EDAC and SAC-N outperform both MSG and PBRL on **hopper**, **walker2d**, and **maze2d** tasks. On **pen-cloned**, **pen-expert**, and **kitchen-mixed** tasks, this relationship is reversed, and on the sparse-reward **antmaze** navigation task, the UQ methods achieve a qualitative improvement (WA methods fail to collect any rewards). In the following section, we investigate the algorithmic decisions behind this gap through targeted ablations of the MSG and PBRL algorithms.

4.2 What are the key design choices for ensemble algorithms?

To answer the second question, we extend our evaluation to include three more environments from the **antmaze** suite. We fix the ensemble size to $N = 10$ and sweep over four values of λ for each algorithm, specified in Appendix A. The results in Table 2 corroborate our earlier findings: WA methods SAC-N and EDAC fail to achieve any reward across all configurations.

We identify independent bootstrap targets as the critical design choice distinguishing MSG and PBRL from the failing baselines. Replacing MSG’s independent targets with shared ones collapses performance to zero across all tasks, regardless of ensemble size – even reducing to $N = 2$ yields only a negligible nonzero score in a single setting. By contrast, the choice of actor loss plays a minor

Table 2: Ablating design choices of ensemble offline RL algorithms on **antmaze** tasks. The entries correspond to mean score computed over 5 training seeds and the standard deviation of the mean. For the different regularization terms of MSG, PBRL, and EDAC, we sweep over four values of λ . Ablation variants are evaluated at the best-performing MSG configuration. **Top:** WA methods with shared bootstrap targets fail entirely, regardless of ensemble size. **Middle:** independent targets are the critical component of MSG – shared targets collapse performance even at $N = 2$, while the LCB actor objective has no significant performance impact. **Bottom:** the form of the regularizer is secondary – with independent bootstrap targets any regularization outperforms all WA methods.

Method	med-diverse	med-play	large-diverse	large-play
EDAC ($N = 10$)	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
SAC ($N = 10$)	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
SAC ($N = 2$)	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
<i>Effect of MSG components</i>				
MSG (reference)	42.88 ± 4.42	37.31 ± 9.86	10.77 ± 3.47	7.12 ± 2.12
MSG, shared ($N \in \{4, 6, 10\}$)	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
MSG, $N = 2$	0.19 ± 0.19	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
MSG, lcb $\rightarrow$ min actor loss	47.69 ± 4.49	56.54 ± 10.90	13.85 ± 5.65	13.08 ± 5.10
<i>Independent targets + effect of regularization</i>				
$\mathcal{R}_{\text{MSG}}$ (reference)	42.88 ± 4.42	37.31 ± 9.86	10.77 ± 3.47	7.12 ± 2.12
$\emptyset$ (no regularization)	3.65 ± 2.79	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
$\mathcal{R}_{\text{PBRL}}$	40.19 ± 13.60	55.58 ± 6.02	2.12 ± 1.15	0.00 ± 0.00
AWAC actor loss	16.54 ± 3.38	55.58 ± 4.30	2.88 ± 1.10	3.46 ± 1.16

role: replacing the LCB operator with a hard minimum improves performance on **medium-diverse** but has no consistent effect elsewhere.

Finally, we ask whether the *form* of regularization matters. On **antmaze**, we find its role is secondary: despite employing qualitatively different regularizers, both MSG and PBRL succeed in solving several tasks, while all shared-target configurations fail completely. Notably, independent targets without any regularization still outperform every shared-target variant, and still remain effective when paired with the BC-regularized actor objective from AWAC as the sole form of regularization.

Table 8 in the Appendix presents ablations for the **pen** tasks. The negative effect of shared targets is weaker there, though the reference MSG variant still achieves the best average performance over all three tasks. On the other hand, the effect of regularizer choice is much more pronounced there, with MSG regularizer being insufficient and the best performance being achieved by the variant with AWAC actor loss. This strengthens the motivation behind our last question: how do independent targets behave in the context of policy-constrained baselines?

4.3 Can independent bootstrap targets improve state-of-the-art policy constraint methods?

To answer the third research question, we investigate whether our insights transfer to the two strongest policy-constrained methods of the evaluation in Section 4.1. To this end, we introduce U-AWAC and U-BRAC, two methods that augment AWAC and ReBRAC with a larger critic ensemble trained utilizing independent targets from Equation 2.

Before evaluating U-AWAC and U-BRAC, we further investigate why sharing bootstrap targets can be detrimental when scaling up the trained critic ensemble. Since AWAC does not explicitly modify Q-value estimates, it provides a clean setting for isolating the effect of target sharing on Q-value bias. To study this effect, we increase the ensemble size of an AWAC agent with shared targets and compare it to U-AWAC with independent targets across three tasks with distinct properties, visualizing both performance and value bias in Figure 2. With shared targets, a sufficiently large ensemble is required to mitigate overestimation bias on **pen-cloned**. However, on tasks with sparse or diverse data, the effect reverses: large ensembles introduce excessive pessimism, leading to complete training collapse. Independent targets produce well-calibrated Q-value estimates across all three tasks for a fixed $N = 10$, outperforming the shared-target method across all tasks and tuning budgets.

We evaluate U-BRAC and U-AWAC using the same protocol described in Section 4.1. We conducted experiments first on sparse-reward navigation tasks and then across the full suite of nine benchmarks.

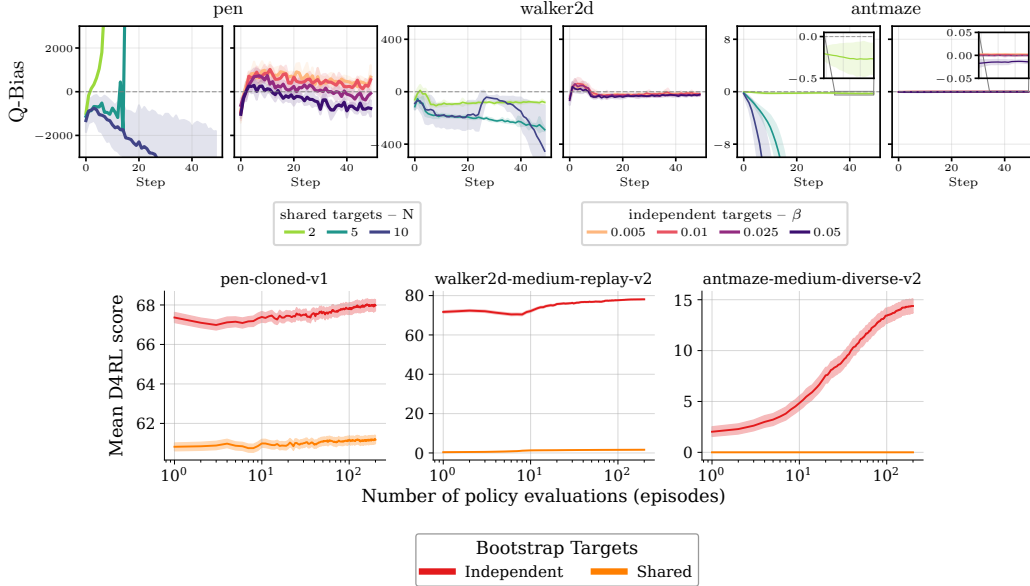


Figure 2: Ablation of shared and independent bootstrap targets for AWAC. We evaluate the effects of increasing the critic ensemble size and using shared or independent bootstrap targets across three tasks. **Top:** Mean Q-value bias across 5 training runs; the shaded region is the standard deviation. Bias is computed as the difference between predicted Q-values and the mean return across several online rollouts. **Bottom:** Final performance using an ensemble size of $N = 10$ on the same three tasks. The independent-target variant (U-AWAC) achieves superior performance on all three tasks.

We specify the hyperparameter spaces used in Table 5. Both methods inherit all hyperparameter settings from their respective base algorithms, with the only modifications being the use of independent bootstrap targets in place of the shared-target critic loss and the adoption of a larger critic ensemble of size $N \in \{10, 30\}$.

Figure 3 shows the performance of the four algorithms across three sparse-reward **antmaze** tasks. We observe that U-BRAC triples ReBRAC’s score on the two **large** mazes and U-AWAC obtains positive rewards across all tasks, whereas AWAC fails to achieve any reward. Figure 4 shows the performance of the same four methods across all nine evaluation tasks and 20 seeds. On five out of nine tasks, either U-AWAC or U-BRAC achieves the strongest performance, with U-AWAC standing out in limited-data domains such as **kitchen-mixed** and **pen-human**, and U-BRAC in the **antmaze** task. However, independent targets appear less effective in settings with highly diverse data – both of the new methods underperform their base counterparts on **maze2d-large** and **walker2d-medium-replay**, suggesting an area for further analysis. In general, independent targets yield a qualitative improvement on sparse-reward tasks, with overall performance remaining comparable (the average best/median policy performance across the entire suite differs by under 2% for ReBRAC/U-BRAC and increases when switching from AWAC to U-AWAC). U-BRAC is the only method to consistently solve **antmaze-large**. Additionally, we compare the proposed methods with all the baselines in Figure 5 in the Appendix.

5 Conclusion

We conducted the first comprehensive evaluation of uncertainty-quantifying (UQ) ensemble methods, MSG and PBRL, against a representative set of policy-constrained and ensemble baselines across a diverse range of tasks using unified methodology. Overall, neither MSG nor PBRL consistently surpassed simpler prior methods based on worst-case aggregation (WA), such as SAC-N and EDAC. Notably, ensemble algorithms were generally outperformed by policy-constrained baselines, with ReBRAC achieving better performance than all ensemble methods on 6 of the 9 considered benchmarks.

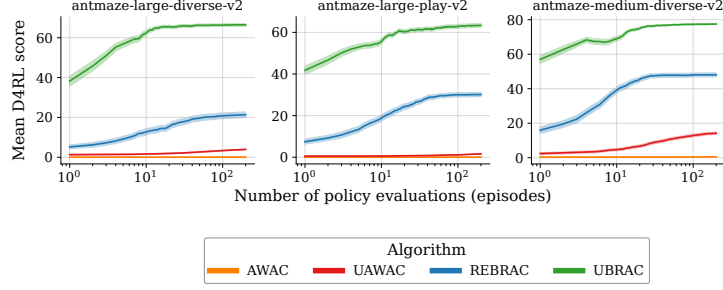


Figure 3: Comparing ReBRAC and AWAC with their independent-ensemble augmented versions, U-BRAC and U-AWAC, across three sparse-reward tasks. U-BRAC triples the performance of ReBRAC, U-AWAC achieves nonzero rewards, whereas AWAC completely fails.

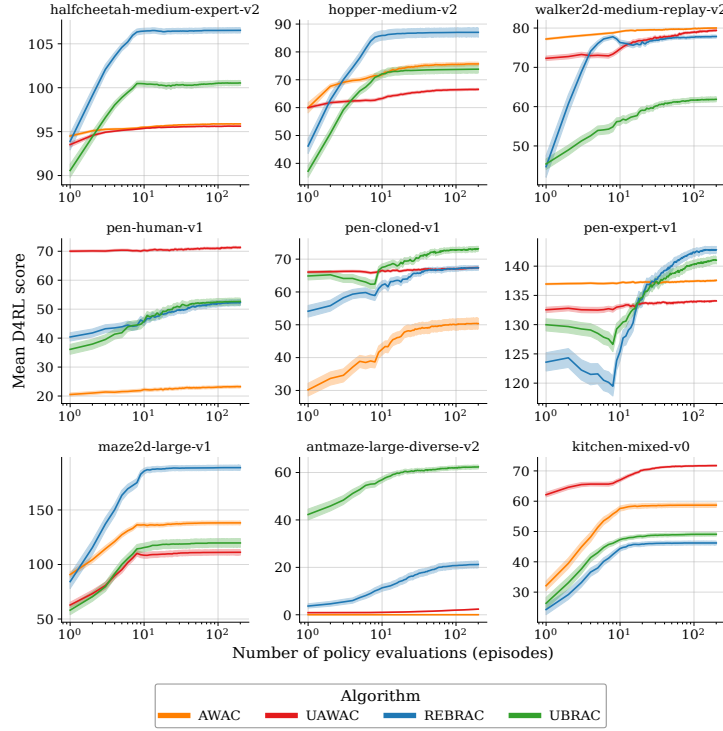


Figure 4: Comparing ReBRAC and AWAC with their independent-ensemble augmented versions, U-BRAC and U-AWAC, across nine evaluation tasks. We train 20 policies for each task.

Nevertheless, MSG and PBRL demonstrated a clear advantage over sparse-reward and Adroit tasks. Through controlled ablations, we identified independently bootstrapped critic targets as the key design factor underlying this improvement, particularly in the sparse-reward context.

Motivated by this we introduced U-AWAC and U-BRAC, which augment two state-of-the-art policy-constrained baselines, AWAC and ReBRAC, with larger critic ensembles trained using independent bootstrap targets. Both methods yield substantial performance gains on sparse-reward tasks while preserving performance across most benchmarks.

One limitation of the proposed approaches is their reduced effectiveness on datasets with highly diverse distributions, such as **maze2d-large** and **walker2d-medium-replay**. A more thorough analysis of the suboptimal behavior remains an important direction for future research.

References

- [1] Gaon An, Seungyong Moon, Jang-Hyun Kim, and Hyun Oh Song. Uncertainty-based offline reinforcement learning with diversified Q-ensemble. *Advances in Neural Information Processing Systems*, 34:7436–7447, 2021.
- [2] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning*, 47(2):235–256, May 2002. ISSN 1573-0565. doi: 10.1023/A:1013689704352. URL <https://doi.org/10.1023/A:1013689704352>.
- [3] Chenjia Bai, Lingxiao Wang, Zhuoran Yang, Zhihong Deng, Animesh Garg, Peng Liu, and Zhaoran Wang. Pessimistic bootstrapping for uncertainty-driven offline reinforcement learning. *International Conference on Learning Representations*, 2022.
- [4] Thomas J DiCiccio and Bradley Efron. Bootstrap confidence intervals. *Statistical science*, 11(3):189–228, 1996.
- [5] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4RL: Datasets for deep data-driven reinforcement learning, 2021. URL <https://arxiv.org/abs/2004.07219>.
- [6] Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. *Advances in Neural Information Processing Systems*, 34:20132–20145, 2021.
- [7] Kamyar Ghasemipour, Shixiang Shane Gu, and Ofir Nachum. Why so pessimistic? Estimating uncertainties for offline RL through ensembles, and why their independence matters. *Advances in Neural Information Processing Systems*, 35:18267–18281, 2022.
- [8] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870. PMLR, 2018.
- [9] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.
- [10] Matthew Thomas Jackson, Uljad Berdica, Jarek Liesen, Shimon Whiteson, and Jakob Nicolaus Foerster. A clean slate for offline reinforcement learning, 2025. URL <https://arxiv.org/abs/2504.11453>.
- [11] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, *International Conference on Learning Representations, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL <http://arxiv.org/abs/1412.6980>.
- [12] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=68n2s9ZJWF8>.
- [13] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems*, 33: 1179–1191, 2020.
- [14] Vladislav Kurenkov and Sergey Kolesnikov. Showing your offline reinforcement learning work: Online evaluation budget matters. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 11729–11752. PMLR, 2022. URL <https://proceedings.mlr.press/v162/kurenkov22a.html>.
- [15] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

- [16] Ashvin Nair, Murtaza Dalal, Abhishek Gupta, and Sergey Levine. Accelerating online reinforcement learning with offline datasets. *CoRR*, abs/2006.09359, 2020. URL <https://arxiv.org/abs/2006.09359>.
- [17] Ian Osband, Charles Blundell, Alexander Pritzel, and Benjamin Van Roy. Deep exploration via bootstrapped DQN. *Advances in Neural Information Processing Systems*, 29, 2016.
- [18] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. John Wiley & Sons, Inc., Hoboken, NJ, USA, 1994. ISBN 978-0-471-61977-2. doi: 10.1002/9780470316887. URL <https://onlinelibrary.wiley.com/doi/book/10.1002/9780470316887>.
- [19] Denis Tarasov, Vladislav Kurenkov, Alexander Nikulin, and Sergey Kolesnikov. Revisiting the minimalist approach to offline reinforcement learning. *Advances in Neural Information Processing Systems*, 36:11592–11620, 2023.
- [20] Denis Tarasov, Alexander Nikulin, Dmitry Akimov, Vladislav Kurenkov, and Sergey Kolesnikov. CORL: Research-oriented deep offline reinforcement learning library. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/62d2cec62b7fd46dd35fa8f2d4aeb52d-Abstract-Datasets_and_Benchmarks.html.
- [21] Jiesheng Wang, Lin Li, Wei Wei, Yujia Zhang, and Xin Yang. Dynamic Uncertainty Estimation for Offline Reinforcement Learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(20):21126–21134, April 2025. ISSN 2374-3468. doi: 10.1609/aaai.v39i20.35410. URL <https://ojs.aaai.org/index.php/AAAI/article/view/35410>.
- [22] Yifan Wu, George Tucker, and Ofir Nachum. Behavior regularized offline reinforcement learning. *CoRR*, abs/1911.11361, 2019. URL <http://arxiv.org/abs/1911.11361>.
- [23] Rui Yang, Chenjia Bai, Xiaoteng Ma, Zhaoran Wang, Chongjie Zhang, and Lei Han. RORL: Robust offline reinforcement learning via conservative smoothing. *Advances in Neural Information Processing Systems*, 35:23851–23866, 2022.
- [24] Chenlu Ye, Rui Yang, Quanquan Gu, and Tong Zhang. Corruption-robust offline reinforcement learning with general function approximation. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 36208–36221. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/71b52a5b3fe2e9303433a174b60e160d-Paper-Conference.pdf.
- [25] Xudong Yu, Chenjia Bai, Hongyi Guo, Changhong Wang, and Zhen Wang. Diverse randomized value functions: A provably pessimistic approach for offline reinforcement learning. *Information Sciences*, 680:121146, 2024.

Technical Appendix

A Experimental Details

We summarize the properties and characteristics of the used D4RL datasets in Table 3.

Table 3: Properties of used D4RL datasets					
Task name	dim(S)	dim(A)	dataset size	horizon	properties
Adroit					
<i>pen-human</i>	45	24	5000	100	high-dimensional, narrow expert data
<i>pen-expert</i>	45	24	500 000	100	
<i>pen-cloned</i>	45	24	500 000	100	
Franka					
<i>kitchen-mixed</i>	60	9	136 950	280	sparse rewards, stitching expert demos
MuJoCo					
<i>halfcheetah-medium-expert</i>	17	6	2 000 000	1000	varied data quality
<i>hopper-medium</i>	11	3	1 000 000	1000	
<i>walker2d-medium-replay</i>	17	6	2 000 000	1000	
Navigation					
<i>antmaze-medium-diverse</i>	29	8	1 000 000	1000	sparse rewards, stitching suboptimal data
<i>antmaze-medium-play</i>	29	8	1 000 000	1000	
<i>antmaze-large-diverse</i>	29	8	1 000 000	1000	
<i>antmaze-large-play</i>	29	8	1 000 000	1000	
PointMaze					
<i>maze2d-large</i>	4	2	4 000 000	800	stitching suboptimal data

Training Hyperparameters We train all of the algorithms over 1 million training steps, utilizing the Adam [11] optimizer. All ensemble methods utilize an ensemble of MLP critics with ReLU activations and three hidden layers, each consisting of 256 neurons. The discount factor $\gamma = 0.99$ and target smoothing coefficient $\tau = 0.005$ [8] are shared across all experiments. All ensemble methods use a squashed-tanh actor, except the two proposed methods, U-AWAC and U-ReBRAC, which retain the policy parametrization of their respective base methods: U-AWAC uses a clipped Gaussian actor, while U-ReBRAC utilizes a deterministic policy. All algorithms that learn a stochastic policy adapt the entropy regularization weight α based on a target entropy [9]. For AWAC and U-AWAC, we set $\alpha = 0$ following the original implementations and Tarasov et al. [20].

Full Evaluation For the full evaluation of Sections 4.1, 4.3, the hyperparameter spaces $\mathcal{H}$ associated with each new algorithm are provided for each algorithm in Table 5. The remaining algorithms use the same settings as in Unifloral [10].

Ablation Study For the ablation study of Section 4.2, we first swept over the hyperparameter grids specified in Table 5. In the follow-up ablations, we select the configuration of MSG with $\lambda_c = 0.1$, which performed the best across all four tasks. We ablated the bootstrap targets to shared, sweeping over $N \in \{2, 4, 6\}$. The entry for independent AWAC is computed with U-AWAC, with the AWR temperature set to a sufficiently high value of 0.01, we found the combination of sufficiently low temperature and independent targets to be the key combination so solve sparse stitching tasks.

Table 4: Hyperparameter settings swept over for the sparse ablation in Section 4.2. We use 5 training seeds per configuration and per task.

Hyperparameter	MSG	PBRL	EDAC	SAC-N	U-AWAC
Ensemble size N	10	10	10	10	10
Independent target weight β	0	0.001	-	-	0.005
Actor reg. weight λ_a	4	-	-	-	-
Critic reg. weight λ_c	{0.1, 0.2, 0.5, 1.0}	{0.01, 0.02, 0.05, 0.1}	{1, 5, 10, 50}	-	-
AWR temperature	-	-	-	-	0.01

Table 5: Hyperparameter search spaces used for full evaluation in Sections 4.1 and 4.3. Multiple values or a range denote a space that is uniformly sampled from during evaluation, while single values are fixed across all training runs. The hyperparameters for the remaining baselines of BC, ReBRAC, SAC-N and EDAC are adopted from Unifloral [10].

Hyperparameter	MSG	PBRL	AWAC	U-AWAC	U-BRAC
<i>Shared optimization</i>					
Critic learning rate α_c	3e-4	3e-4	3e-4	3e-4	1e-3
Actor learning rate α_a	1e-4	3e-4	3e-4	3e-4	1e-3
Batch size	256	256	256	256	1024
Discount γ	0.99	0.99	0.99	0.99	0.99
Polyak step size τ	0.005	0.005	0.005	0.005	0.005
<i>Ensemble & target scheme</i>					
Ensemble size N	{10, 30}	{10, 30}	2	{10, 30}	10
Shared targets	No	No	Yes	No	No
<i>Ensemble Regularization</i>					
Critic regularizer	$\mathcal{R}_{\text{MSG}}$	$\mathcal{R}_{\text{PBRL}}$	None	None	None
Critic reg. weight λ_c	{0, 0.1, 0.5, 1, 2, 4, 8}	{0.01, 0.2}	-	-	-
Actor reg. weight λ_a	{4, 8} -	-	-	-	-
<i>Independent-target weights</i>					
β_{id}	0	{0.0001, 0.01}	-	{0.005, 0.01, 0.025, 0.05}	{0.0005, 0.001}
<i>Policy improvement objective</i>					
Operator	LCB	Min	AWR	AWR	TD3+BC
AWR temperature	-	-	{0.01, 0.33, 1.0}	{0.01, 0.33, 1.0}	-
Actor BC weight	-	-	-	-	[5e-4-1]
Critic BC weight	-	-	-	-	0

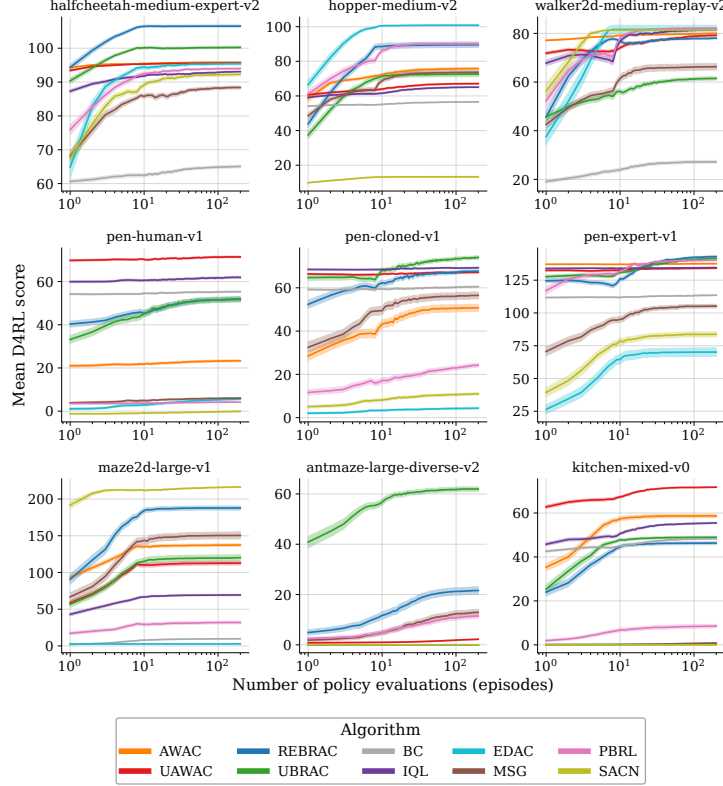


Figure 5: Comparison of U-BRAC and U-AWAC against all evaluated ensemble methods across $P = 20$ trained policies.

B Additional Figures and Tables

In this appendix, we include additional evaluation plots and numerical results. Figure 5 presents the evaluation results of all ten considered algorithms across all of the nine benchmarks. Tables 6 and 7 report the mean and standard deviation of the scores achieved by the best and median (eleventh-best) trained policies for each algorithm over 1000 rollouts.

Broader Hyperparameter Sweep for PBRL We further evaluated PBRL over a broader hyperparameter space. The performance remained largely similar across tasks, indicating that our insights are robust with respect to different hyperparameter choices. Figure 6 compares the results of this sweep with the main evaluation.

Specifically, we considered lower values of the regularization parameter $\lambda_C \in \{0.001, 0.01, 0.1\}$, $\beta \in \{0.0001, 0.001, 0.01\}$, and ensemble size $N \in \{10, 30\}$.

Dataset	AWAC	BC	EDAC	IQL	MSG	PBRL	REBRAC	SACN	UREBRAC	UAWAC
antmaze-large-diverse-v2	0.20 ± 4.47	0.00 ± 0.00	0.00 ± 0.00	0.10 ± 3.16	29.90 ± 45.78	23.80 ± 42.59	41.00 ± 49.18	0.00 ± 0.00	70.40 ± 45.65	4.50 ± 20.73
halfcheetah-medium-expert-v2	96.14 ± 3.19	70.76 ± 23.79	99.34 ± 3.52	93.72 ± 5.33	92.37 ± 8.03	95.83 ± 4.87	110.89 ± 4.88	96.85 ± 10.65	103.99 ± 2.22	96.01 ± 3.82
hopper-medium-v2	84.95 ± 16.24	57.74 ± 9.59	101.70 ± 0.24	69.29 ± 15.79	86.79 ± 17.16	101.17 ± 6.98	101.07 ± 8.63	15.29 ± 0.86	92.17 ± 13.53	71.71 ± 16.97
kitchen-mixed-v0	67.42 ± 18.86	50.50 ± 11.72	0.65 ± 3.98	59.70 ± 13.45	2.30 ± 7.65	19.48 ± 10.37	51.45 ± 13.98	0.00 ± 0.00	53.12 ± 13.72	73.28 ± 7.92
maze2d-large-v1	155.54 ± 49.55	14.01 ± 7.87	5.92 ± 47.83	82.47 ± 52.67	198.68 ± 48.87	65.52 ± 106.88	205.30 ± 44.67	218.65 ± 49.57	174.49 ± 70.31	135.06 ± 42.57
pen-cloned-v1	71.74 ± 62.73	69.08 ± 63.10	12.80 ± 43.74	74.92 ± 64.39	79.17 ± 74.97	33.12 ± 66.45	74.12 ± 65.02	18.37 ± 52.87	83.69 ± 71.66	73.41 ± 62.59
pen-expert-v1	140.11 ± 35.14	119.27 ± 50.24	93.03 ± 70.32	137.70 ± 36.59	114.66 ± 61.81	142.39 ± 33.51	148.37 ± 28.13	98.98 ± 70.33	147.24 ± 27.35	136.96 ± 35.42
pen-human-v1	28.60 ± 57.42	60.75 ± 58.77	15.13 ± 39.09	69.99 ± 61.47	10.82 ± 43.04	6.97 ± 25.69	66.21 ± 65.78	2.30 ± 21.76	67.68 ± 64.59	77.81 ± 62.40
walker2d-medium-replay-v2	81.07 ± 5.49	37.21 ± 30.44	88.71 ± 0.51	84.76 ± 16.30	84.25 ± 15.18	83.48 ± 5.83	81.60 ± 22.16	81.65 ± 0.69	72.69 ± 28.19	82.29 ± 13.51
Average	80.64 ± 28.12	53.26 ± 28.39	46.36 ± 23.25	74.74 ± 29.91	77.66 ± 35.83	63.53 ± 33.69	97.78 ± 33.60	59.12 ± 22.97	96.16 ± 37.47	83.45 ± 29.55

Table 6: Performance of the best policy trained by each algorithm in Sections 4.1, 4.3

Dataset	AWAC	BC	EDAC	IQL	MSG	PBRL	REBRAC	SACN	UREBRAC	UAWAC
antmaze-large-diverse-v2	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	30.30 ± 45.96	0.00 ± 0.00
halfcheetah-medium-expert-v2	95.25 ± 3.18	62.42 ± 23.73	86.59 ± 4.17	87.59 ± 16.22	77.20 ± 24.45	79.88 ± 22.53	93.58 ± 16.70	77.26 ± 27.15	93.57 ± 2.05	94.14 ± 2.31
hopper-medium-v2	65.02 ± 12.00	54.52 ± 8.47	57.27 ± 9.73	58.42 ± 12.55	52.28 ± 10.69	54.35 ± 25.94	56.46 ± 9.73	9.81 ± 0.09	48.04 ± 9.88	59.06 ± 13.38
kitchen-mixed-v0	25.00 ± 2.24	44.12 ± 15.81	0.00 ± 0.00	47.98 ± 14.97	0.00 ± 0.00	0.00 ± 0.00	8.75 ± 14.00	0.00 ± 0.00	3.55 ± 9.99	64.75 ± 12.45
maze2d-large-v1	79.33 ± 19.32	-1.54 ± 3.64	2.70 ± 37.65	41.33 ± 16.55	39.60 ± 8.29	4.10 ± 9.28	46.72 ± 10.20	215.62 ± 53.83	45.51 ± 15.87	48.74 ± 16.43
pen-cloned-v1	21.08 ± 43.93	59.54 ± 62.84	1.94 ± 11.66	68.09 ± 62.16	24.03 ± 46.36	-0.21 ± 13.46	57.40 ± 65.26	2.46 ± 25.99	63.44 ± 64.05	66.05 ± 62.76
pen-expert-v1	137.09 ± 34.48	111.04 ± 54.83	2.72 ± 27.81	133.71 ± 38.18	44.81 ± 58.23	136.04 ± 42.32	126.24 ± 48.74	2.27 ± 25.15	128.42 ± 42.94	133.72 ± 38.46
pen-human-v1	22.35 ± 51.52	53.39 ± 58.05	-0.36 ± 10.74	59.56 ± 59.54	3.28 ± 26.48	3.74 ± 27.70	42.74 ± 59.71	-1.51 ± 5.69	25.09 ± 50.52	70.37 ± 61.99
walker2d-medium-replay-v2	77.16 ± 11.97	16.07 ± 18.71	-0.12 ± 0.01	73.39 ± 20.93	47.99 ± 18.36	51.36 ± 25.06	63.54 ± 30.00	78.61 ± 0.18	45.06 ± 32.46	74.46 ± 18.91
Average	58.03 ± 19.85	44.40 ± 27.34	16.75 ± 11.31	63.34 ± 26.79	32.13 ± 21.43	36.58 ± 18.48	55.05 ± 28.26	42.72 ± 15.34	53.66 ± 30.41	67.92 ± 25.19

Table 7: Performance of the median policy trained by each algorithm in Sections 4.1, 4.3

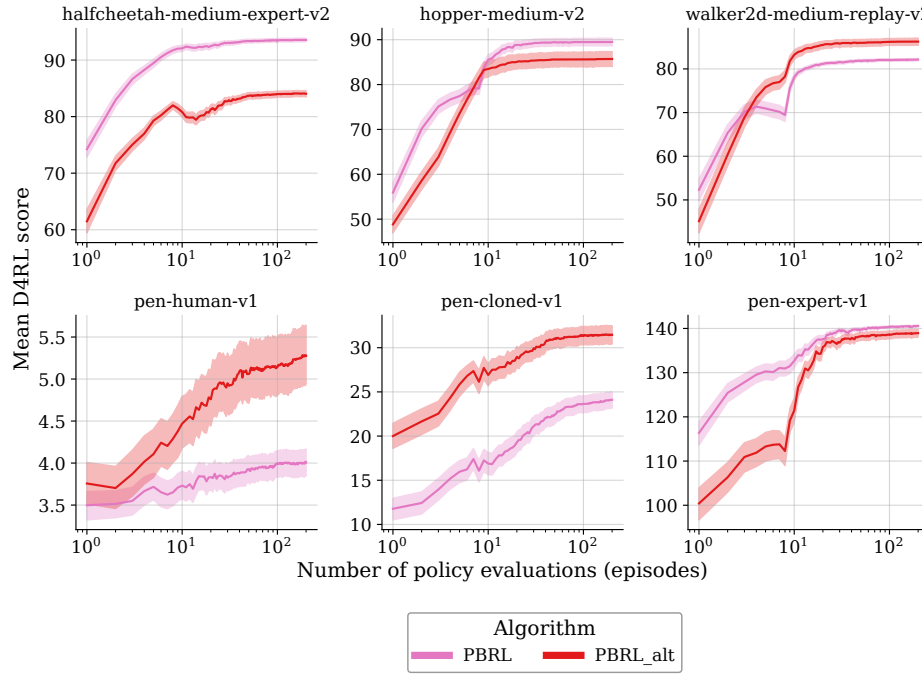


Figure 6: Evaluation of PBRL under a broader hyperparameter sweep. Performance is comparable to the main results across the tested tasks.

Table 8: Ablating design choices of ensemble offline RL algorithms on the **Adroit pen** tasks. Entries are mean normalized scores over 5 training seeds and the standard deviations of the means.

Method	pen-cloned	pen-expert	pen-human	Avg.
<i>Worst-case aggregation baselines</i>				
EDAC ($\lambda = 1.0$)	0.65 ± 4.26	2.14 ± 2.74	-1.86 ± 0.68	0.31
EDAC ($\lambda = 5.0$)	-1.45 ± 1.56	-1.72 ± 1.81	0.46 ± 5.58	-0.90
EDAC ($\lambda = 10.0$)	5.25 ± 4.40	-2.11 ± 1.00	-1.68 ± 1.39	0.49
EDAC ($\lambda = 50.0$)	-1.20 ± 1.45	1.78 ± 3.42	0.23 ± 5.85	0.27
EDAC ($\lambda = 100.0$)	0.84 ± 4.24	-1.03 ± 2.41	2.53 ± 7.77	0.78
SAC-N ($N = 2$)	-1.51 ± 1.35	-2.85 ± 0.99	2.46 ± 4.03	-0.63
SAC-N ($N = 10$)	1.51 ± 4.95	-0.04 ± 2.63	-1.34 ± 1.00	0.04
<i>MSG target and actor ablations</i>				
MSG (reference)	4.71 ± 3.58	2.36 ± 1.70	4.29 ± 6.04	3.79
MSG, min actor loss ($\beta = 0.0, \lambda_c = 0.1$)	-1.47 ± 2.14	0.67 ± 3.94	1.95 ± 3.46	0.38
MSG, shared ($N = 2, \beta = 0.0, \lambda_c = 0.1$)	2.78 ± 1.63	3.25 ± 2.00	2.97 ± 0.76	3.00
MSG, shared ($N = 4, \beta = 0.0, \lambda_c = 0.1$)	3.06 ± 3.30	-2.36 ± 1.34	-1.31 ± 1.81	-0.20
MSG, shared ($N = 6, \beta = 0.0, \lambda_c = 0.1$)	2.06 ± 6.13	2.96 ± 3.17	-2.26 ± 1.51	0.92
MSG, shared ($N = 10, \beta = 0.0, \lambda_c = 0.1$)	9.76 ± 5.38	2.86 ± 5.67	-2.56 ± 0.95	3.35
<i>Independent targets + effect of regularization</i>				
$\mathcal{R}_{\text{MSG}}$ (reference)	4.71 ± 3.58	2.36 ± 1.70	4.29 ± 6.04	3.79
MSG, no reg ($\beta = 0.01, \lambda_c = 1.0$)	2.83 ± 3.31	11.15 ± 5.82	-0.92 ± 2.08	4.35
$\mathcal{R}_{\text{PBRL}}$ ($\beta = 0.0001, \lambda_c = 0.01$)	31.49 ± 10.84	58.10 ± 9.33	7.11 ± 5.33	32.23
$\mathcal{R}_{\text{PBRL}}$ ($\beta = 0.001, \lambda_c = 0.01$)	32.25 ± 10.26	67.33 ± 3.84	5.34 ± 7.86	34.97
$\mathcal{R}_{\text{PBRL}}$ ($\beta = 0.001, \lambda_c = 0.02$)	30.10 ± 4.51	87.94 ± 8.99	9.07 ± 7.65	42.37
$\mathcal{R}_{\text{PBRL}}$ ($\beta = 0.001, \lambda_c = 0.05$)	11.66 ± 6.56	117.86 ± 9.12	3.58 ± 8.01	44.37
$\mathcal{R}_{\text{PBRL}}$ ($\beta = 0.001, \lambda_c = 0.1$)	0.20 ± 1.97	132.73 ± 6.39	1.43 ± 0.70	44.79
AWAC actor loss ($\lambda_c = 1.0, T_{\text{AWR}} = 0.01$)	65.81 ± 5.03	129.69 ± 3.56	70.91 ± 5.36	88.80